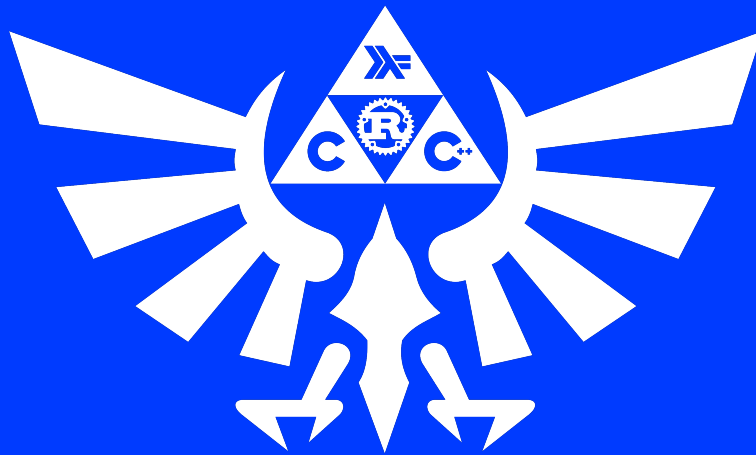




DAY 01

< C, LIFE, THE UNIVERSE AND EVERYTHING
ELSE />



DAY 01

All your exercises will be compiled using `epiclang` with the `-std=gnu17 -Wall -Wextra` flags, unless specified otherwise. Memory errors or leaks are **not** accepted. All `libc` functions are allowed.



Every function implemented in a header or any unprotected header leads to 0 for the exercise.



To avoid compilation problems during automated tests, please include all necessary files within your headers.

Unit Tests

It is highly recommended to test your functions as you implement them. It is common practice to create and use what are called **unit tests**.

From now on, we expect you to write unit tests for your functions (when possible). To do so, please follow the instructions in the **"How to write Unit Tests"** document on the intranet, available [here](#).

Create a directory named `tests`. For each of the functions you turn in, create a file in that directory named `tests_FUNCTION_NAME.c` containing all the tests needed to cover all of the exercise's possible cases (regular or irregular).

Here is a sample set of unit tests for the **my_strlen** function :

```
#include <criterion/criterion.h>

Test(my_strlen, positive_return_value)
{
    cr_assert_eq(my_strlen("toto"), 4);
}

Test(my_strlen, empty_string)
{
    cr_assert_eq(my_strlen(""), 0);
}
```

Exercise -1 - Family reunion

Hello, nice to see you all again.

Now, as the more perceptive of you have probably realised by now, this is **Hell**, and I am the **Devil**. Good evening. You can call me **Toby**, if you like – we try and keep things informal here, as well as **infernal**.

That's just a little joke.

✓ [Rowan Atkinson Live](#), 1993.

Before diving into discovering your work, we would like you to do one thing :

Join the official [Discord server of Epitech](#) to get in touch with all other students and national staff members.



When you join the Discord server, go to the #commands channel and type /login to link your Discord account with your Epitech account.

The module staff will be glad to (sometimes) answer your questions in the #pdg channel of your promotion. Feel free to share your thoughts with others, or ping us if you spot an error or a typo in a subject.

Exercise 0 - Z is for Zorglub



Turn in : `Makefile`, `z.c` in `zorglub/`

Compilation : using your `Makefile`

Executable name : `z`

Our Lord and Genius Zorglub, master of Zorgland, wants you to design the software for his new brain-washing machine to zorgmanize his enemies into zorgmen. The machine works by sending a specific character into the mind of its victim. The character must be chosen carefully using an algorithm devised by our Lord and Genius Zorglub.

The algorithm uses the prisoner ID to determine which character should be used. An ID is a hexadecimal number that fits in a `uint64_t`, given as parameter to your program. You must then display the character to be used, following the rules defined by the Grand Z :

- ✓ If the binary notation of the ID is a palindrome, it means that the victim is just a dummy used for testing purpose. You must display the default brain-washing byte `0172` (octal) and validation byte `012` (octal).
- ✓ If the ID is a multiple of 13 and 29 and 89 or 41 and 71 or 67 or 7 and 43 and 47 and 53, you must display the string `"z\n"` and immediately stop your program.
- ✓ If the ID is `0x12345678`, `0x87654321`, `0x011111010` or `0x01011010`, you must display the following bits :
`0111101000001010`.
- ✓ If one of the bytes of the ID is worth `0x42`, it means that the victim is tough and the strongest character must be used : `'z'`. The byte `0x0A` is then displayed.
- ✓ If the last byte of the ID is null, the victim is just a minion of our enemies. As we don't care about them, we will brain-wash them using the last letter of the alphabet in lowercase. The output will then be flushed with a line feed.
- ✓ If the ID is a multiple of the sum of its 8 bytes, you must display the sixth character of the ASCII table starting from the end, followed by a `'\n'`.
- ✓ If the 8 bytes of the ID are identical, it means that the prisoner is one of our special guest. You will use the character `'z'` in lowercase, and your program will end with a new line. This case overrides all the other cases.
- ✓ If the parameter contains non-hexadecimal characters, this is an error. The program must handle this case by displaying the error character `'z'` followed by a line feed.
- ✓ If the parameter does not fit in a `uint64_t`, this is an error. In this very specific case, the very specific character `'z'` must be used to brain-wash the victim, followed by the tenth character of the ASCII table.
- ✓ If there is no argument, the ID to be used is the return value of the function `time(NULL)`.
- ✓ If there is too many arguments, your program must use the first valid argument starting from the end. If no argument is valid, we consider there is no argument and use the preceding rule.

Your program must *always* return 0 as it will never fail, even in case of invalid input.

```
Terminal
$> make
[...]
$> ./z | cat -e
z$
$> ./z "0x01" "0x02" "0x03" "0x04" | cat -e
z$
$> ./z "0x42242112" | cat -e
z$
$> ./z "0xABCDDCBA" | cat -e
z$
$> ./z "0x0" | cat -e
z$
$> ./z "0x5928AB00" | cat -e
z$
$> ./z "0xABCDEFGH" | cat -e
z$
$> ./z "0xBEBECACA" | cat -e
z$
$> ./z "0xTR0L0L0L" | cat -e
z$
$> ./z "still_reading?" | cat -e
z$
$> ./z "invalid_ID" ; echo $?
z
0
```



Our Lord and Genius Zorglub is expecting you to complete this simple task in less than 2 minutes.

Exercise 1 - String module



Turn in : `Makefile`, `string.h`, `*.c` in `string/`

Compilation : using your `Makefile`

Executable name : `libstring.a`

Create a `string_t` module.

Your module should contain an initialization function and a destruction function with the following prototypes :

```
1. void string_init(string_t *this, const char *s);
2. void string_destroy(string_t *this);
```

`string_init` assigns the value of `s` to the internal string of the structure `string_t` and initialize its internal function pointers.

Add the following **member function** to your module :

```
1. bool empty(const string_t *this);
```

Returns `true` if the string is empty, `false` otherwise. An empty string is a string with a length of 0.

```
2. size_t length(const string_t *this);
```

Returns the size of the string.

```
3. char at(const string_t *this, size_t pos);
```

Returns the `char` at the `pos` position of the current instance, or -1 if the position is invalid. The terminating `'\0'` is not considered as being part of the string.

```
4. void print(const string_t *this);
```

Displays the content of the current instance to the standard output. Be careful, we **never** said anything about **carriage returns**!

```
5. const char *data(const string_t *this);
```

Returns the `char *` buffer contained in the current instance.

```
6. void clear(string_t *this);
```

Empties the content of the current instance.

```
7. void assign(string_t *this, const char *str);
```

Sets the content of the current instance to that of `str`.

```
8. void append(string_t *this, const char *str);
```

Appends the content of `str` to that of the current instance.



Memory allocation failure must `abort` the program. See `man 3 abort`.

Here is a sample `main` function and its expected output :

```
#include "string.h"

int main(void)
{
    string_t s;

    string_init(&s, "Foo");
    s.append(&s, "Bar\n");
    s.print(&s);
    s.clear(&s);
    s.append(&s, "Hello");
    s.append(&s, " ");
    s.append(&s, "World\n");
    s.print(&s);
    string_destroy(&s);
    return (0);
}
```

```
Terminal
$> make
[...]
$> epiclang -std=gnu17 -Wall -Wextra -iquote . test_main.c -L. -lstring
$> ./a.out | cat -e
FooBar$
Hello World$
```



Be careful with memory leaks and memory errors.



Here's some bonus methods ideas:

```
int to_int(const string_t *this)
void insert(string_t *this, size_t pos, const char *str);
int compare(const string_t *this, const char *str);
int find(const string_t *this, const char *str, size_t pos);
string_t *substr(const string_t *this, int offset, int length);
string_t **split(const string_t *this, char separator);
size_t copy(const string_t *this, char *s, size_t n, size_t pos);
...and many more.
```


Exercise 2 - Array module



Turn in : `Makefile`, `array.h`, `*.c` in `array/`
Compilation : using your `Makefile`
Executable name : `libarray.a`

Create an `array_t` module that handles a generic `void *` pointers array.

Your module should contain an initialization function and a destruction function with the following prototypes :

```
1. void array_init(array_t *this, size_t size);
2. void array_destroy(array_t *this);
```

`array_init` initializes its internal array with `size` elements set to `NULL` and initialize its internal functions pointers.

Add the following **member function** to your module :

```
1. bool empty(const array_t *this);
```

Returns `true` if the array is empty, `false` otherwise.

```
2. size_t size(const array_t *this);
```

Returns the size of the array.

```
3. void resize(array_t *this, size_t size);
```

Resize array to `size` elements, new elements are initialized to `NULL`.

```
4. void *get(const array_t *this, size_t index);
```

Return the pointer at `index` position, or `NULL` if out of bounds.

```
5. bool set(array_t *this, size_t index, void *value);
```

Set the pointer at `index` position and returns `true`, or does nothing and returns `false` if `index` out of bounds.

```
6. void apply(array_t *this, void *(*function)(void *));
```

For each element of the array, call the `function` with the element and replace its value with the return value of `function`.

Don't forget to create your own `main` function to test your module, or even better : Criterion tests !



Here's some bonus methods ideas:

```
int sort(array_t *this, int(*compare)(void *, void *));  
void push(array_t *this, void *value);  
void pop(array_t *this, void *value);  
...and many more.
```

Exercise 3 - List module



Turn in : `Makefile`, `list.h`, `*.c` in `list/`
Compilation : using your `Makefile`
Executable name : `liblist.a`

Create a `list_t` module that handle a generic `void *` pointers list.

Your module should contain an initialization function and a destruction function with the following prototypes :

```
1. void list_init(list_t *this);  
2. void list_destroy(list_t *this);
```

`list_init` initializes its internal list as empty, and initialize its internal functions pointers.

Add the following **member function** to your module :

```
1. bool empty(const list_t *this);
```

Returns `true` if the list is empty, `false` otherwise.

```
2. size_t size(const list_t *this);
```

Returns the number of elements in the list.

```
3. void *get(const list_t *this, size_t position);
```

Get the element at the given `position` in the list, returns `NULL` if out of bounds.

```
4. bool set(list_t *this, size_t position, void *value);
```

Set the pointer at `position` and return `true`, or does nothing and return `false` if `position` out of bounds.

```
5. void push_front(list_t *this, void *value);
6. void push_back(list_t *this, void *value);
```

Push the `value` at the front/back of the list.

```
7. void pop_front(list_t *this);
8. void pop_back(list_t *this);
```

Remove the first/last element of the list, does nothing if the list is empty.

```
9. void clear(list_t *this);
```

Remove every element of the list.

```
10. void apply(list_t *this, void (*function)(void *));
```

For each element of the list, call the `function` with the element and replace its value with the return of `function`.

Don't forget to create your own `main` function to test your module, or even better : Criterion tests !



Here's some bonus methods ideas:

```
void push_position(list_t *this, size_t position, void *value);
void pop_position(list_t *this, size_t position);
void sort(list_t *this, int(*compare)(void *, void *));
void remove_if(list_t *this, int(*compare)(void *, void *));
...and many more.
```

v 4.0.1

{EPITECH}